

软件测试实验报告

实验名称：实验一 初步探索软件测试	实验日期：2026年9月10日
姓名：_____	学号：_____
班级：_____	被测语言版本：C++/Qt (build-qt)

1. 缺陷记录报告

测试对象与范围：C++/Qt 科学计算器（build-qt/QtDemo.exe）。本次以按钮输入和 Calculator 核心逻辑为主，覆盖基本四则运算、常数、科学函数、边界输入和错误处理。以下结果均可由给出的输入序列复现。

【功能错误】常数与数字直接拼接时被当作普通十进制数字

1. 缺陷简述：输入数字后直接输入 π ，程序没有按数学计算器的常见规则补充乘法，而是把 π 展开后的数值与前面的数字连成一个新的十进制数。
2. 复现步骤：点击“AC”；依次输入 2、 π ；点击“=”；记录显示结果。
3. 预期结果：程序应将 2π 识别为 $2 \times \pi$ ，结果约为 6.2831853072；或者在不支持隐式乘法时，明确提示输入格式错误。
4. 实际结果：显示 23.1415926536。该结果相当于把 π 替换为 3.141592653589793 后拼成 23.141592653589793，而不是计算 $2 \times \pi$ 。
5. AI 提示词：请审查这个 C++ 科学计算器对数字、 π 、e 的连续输入，设计能发现隐式乘法或常数拼接问题的测试用例，并给出预期结果。

【功能错误】嵌套数学函数无法正常计算

1. 缺陷简述：科学函数的参数通过正则表达式匹配，遇到嵌套括号时不能完整识别，导致合法的嵌套表达式被判定为错误。
2. 复现步骤：点击“AC”；依次点击 sin、sqrt，输入 9，再输入两个右括号；点击“=”。等价表达式为 $\sin(\sqrt{9})$ 。
3. 预期结果：在角度模式下应先计算 $\sqrt{9}=3$ ，再计算 $\sin(3^\circ)$ ，结果约为 0.0523359562。
4. 实际结果：显示“错误”，随后被自动清除。源码中的正则表达式使用 `[^)]`，只截取到第一个右括号，无法处理嵌套函数。
5. AI 提示词：请针对支持 sin、cos、tan、sqrt、log 等函数的计算器，设计嵌套函数、括号匹配和组合表达式测试，并判断哪些结果应当合法。

【输入校验错误】非整数阶乘被静默当作整数阶乘计算

1. 缺陷简述：阶乘定义域通常为非负整数，但程序接受小数输入；例如 3.5! 没有报错，也没有给出伽马函数意义下的结果，而是按循环次数得到 3!。
2. 复现步骤：点击“AC”；点击 n!；依次输入 3、.、5；输入右括号；点击“=”。
3. 预期结果：程序应拒绝非整数阶乘并显示清晰的输入错误，或明确声明采用伽马函数扩展并计算约 11.6317。
4. 实际结果：显示 6。源码的 factorial 函数使用 int i 从 2 循环到 $i \leq n$ ，3.5 只执行到 3，因而错误地返回 3!。
5. AI 提示词：请审查阶乘按钮的定义域和边界值，至少覆盖 0、1、负数、小数、大整数，并区分应计算、应拒绝和应提示溢出的情况。

【输入问题】重复小数点未在输入阶段拦截

1. 缺陷简述：同一数字中可以连续输入多个小数点，界面没有即时阻止或提示，用户只能在点击等号后才发现表达式无效。
2. 复现步骤：点击“AC”；依次输入 1、.、.、2；点击“=”。
3. 预期结果：第二个小数点应被忽略或提示“一个数字只能包含一个小数点”，用户仍可继续编辑有效表达式。
4. 实际结果：输入阶段显示 1..2；点击“=”后显示“错误”，并在约 2 秒后自动清除。
5. AI 提示词：请为计算器的小数点输入设计等价类和边界测试，检查重复小数点、以小数点开头、连续运算符和空括号等非法输入的反馈。

2. 思考与问答

问题1：为什么需要软件测试？软件不测试导致的后果是什么？

软件测试的价值在于用可重复的方式验证软件是否满足需求，并尽早发现功能、边界、异常处理和交互方面的问题。计算器看起来功能简单，但输入组合很多，普通演示只能覆盖少量“正常路径”，不能证明表达式解析、数学定义域和错误提示都正确。若软件未经测试就交付，轻则出现结果错误、程序状态被清空、用户无法判断如何修正；重则错误数据会继续传递到报表、交易或控制系统，造成经济损失和安全风险。测试还可以把“预期行为”明确下来，使开发者能够定位缺陷并回归验证修复效果。因此，软件测试不是为了证明程序绝对没有错误，而是通过风险分析提高软件质量与使用信心。

问题2：简要写写对该计算器进行测试的思路？

我先确认运行环境和界面按钮是否可用，再按由浅入深的顺序测试：第一步测试数字输入、四则运算、清除、退格和键盘输入等基本功能；第二步测试运算符优先级、括号、幂运算和连续计算；第三步测试 π 、 e 、三角函数、对数、平方根、绝对值和阶乘，并分别切换角度/弧度模式；第四步使用边界值和等价类测试，例如 0、负数、小数、超大数、除以 0、负数开方、重复小数点、空括号和嵌套函数；第五步检查错误提示、错误后的自动清除、历史表达式和界面显示。每个用例都记录输入、预期结果和实际结果，发现问题后用最短步骤复现，再结合源码定位可能原因，最后对已发现缺陷做回归测试。

问题3：本次实验的收获？

本次实验让我认识到，高效测试的关键不是盲目增加操作次数，而是围绕功能模型和风险设计有代表性的用例。等价类可以减少重复输入，边界值更容易暴露定义域和数值处理问题，源码阅读则能帮助我把“现象”进一步联系到正则匹配、循环条件和状态变量。AI

适合辅助生成测试维度、补充容易遗漏的组合和解释异常现象，但不能直接替代运行验证；我需要把 AI 的建议转化为具体输入，检查实际结果，并删除没有证据支持的结论。本次还体会到缺陷报告必须写清复现前置条件、逐步操作、预期与实际差异，这样开发者才能快速定位和修复。